Summer School on AI for Economics and Finance

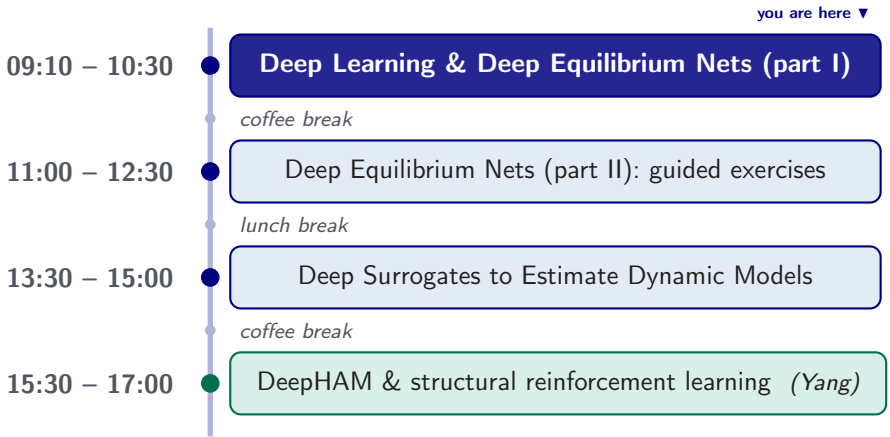
Introduction to Machine Learning and Deep Learning

Simon Scheidegger

HEC, University of Lausanne

University of Torino
August 24, 2026

Day 1 at a Glance



Materials: github.com/sischei/summer-school-AI-. . .-2026 · **Cloud:** nuvolos.cloud

Roadmap of This Lecture (45–50 min)

Setting the Stage (6 min)

- Why deep learning, why now?
- Deep learning in economics and finance

Machine Learning Foundations (8 min)

- Supervised, unsupervised, RL
- The three-step ML recipe

From Perceptrons to Deep Nets (8 min)

- Neurons, activations, universal approximation

Training Neural Networks (12 min)

- Losses, (stochastic) gradient descent
- Backpropagation, optimizers

Generalization & Regularization (6 min)

- Overfitting, early stopping, dropout

Outlook & Practice (6 min)

- Beyond feedforward nets; software; exercises

The Rise of Neural Networks

TOM SIMONITE BUSINESS 01.25.19 01:05 PM

DEEPMIND BEATS PROS AT STARCRAFT IN ANOTHER TRIUMPH FOR BOTS

A pro gamer and an AI bot duke it out in the strategy game StarCraft, which has become a test bed for research on artificial intelligence. STARCRAFT

IN LONDON LAST month, a team from Alphabet's UK-based artificial intelligence research unit DeepMind quietly laid down a new marker in the contest between humans and computers. On Thursday it revealed the achievement in a three-hour YouTube stream, in which aliens and robots fought to the death.

TOM SIMONITE BUSINESS 10.18.17 01:00 PM

THIS MORE POWERFUL VERSION OF ALPHAGO LEARNS ON ITS OWN

NOAH SHELDON FOR WIRED

AT ONE POINT during his historic defeat to AlphaGo last year, world champion Go player Lee Sedol abruptly left the room. The bot had played a move that confounded established theories of the board game, a moment that came to epitomize the mystery surrounding AlphaGo.

NEWS BIOLOGY 21 DECEMBER 2017

AI beats docs in cancer spotting

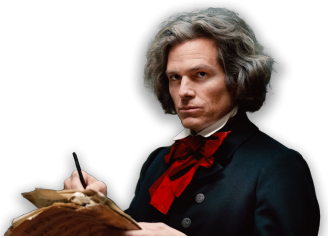
A new study provides a fresh example of machine learning as an important diagnostic tool. Paul Biegler reports.

Deep Learning Software Speeds Up Drug Discovery

Wed, 01/16/2019 - 8:00am 1 Comment by Kenny Walter , Science Reporter - @RandDMagazine

The long, arduous process of narrowing down millions of chemical compounds to just a select few that can be further developed into mature drugs, may soon be shortened, thanks to new artificial intelligence (AI) software.

AI Music Generation: Beethoven's Unfinished 10th



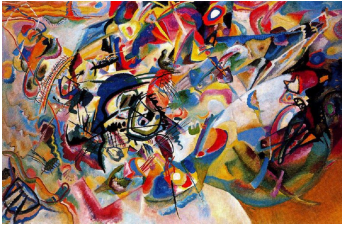
Beethoven X, The AI Project: musicologists, composers and AI researchers trained a network on Beethoven's complete works and sketches, then had it generate the second and third movements of his unfinished 10th symphony.

- The network proposes continuations *in Beethoven's style*; human experts curate and orchestrate.
- World premiere: Bonn, 9 October 2021. Reception was mixed, which is itself a good prompt: *what does the model actually learn?*

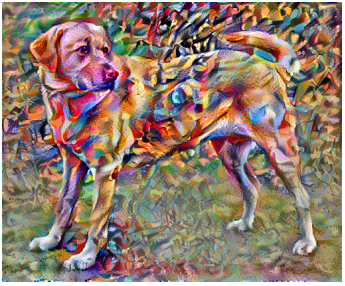
Images and project: Beethoven X, The AI Project, beethovenx-ai.com

Neural Style Transfer

Content image



Style image (Kandinsky)



Result

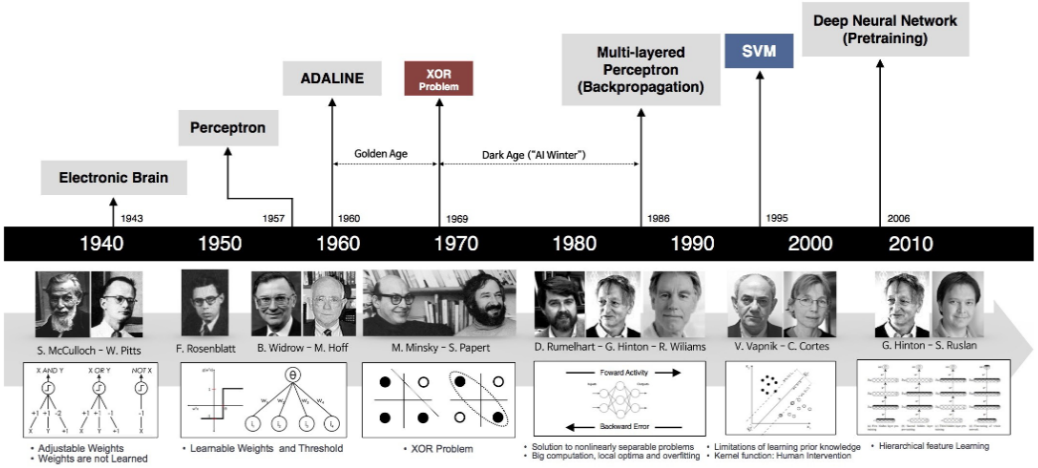
Gatys, Ecker & Bethge (2015): a neural network separates and recombines content and style of images.

Self-Driving Cars

ALVINN (1989), one of the first autonomous vehicles, using a simple neural network.

Waymo (2017+), deep learning for perception, prediction, and planning.

A Timeline of Deep Learning



Deep Learning in Economics and Finance

Forecasting & nowcasting

- GDP nowcasting, inflation, yield curves

Financial stability & supervision

- Fraud detection, credit risk, stress testing

Text & communication

- Central-bank speeches, news sentiment, policy uncertainty

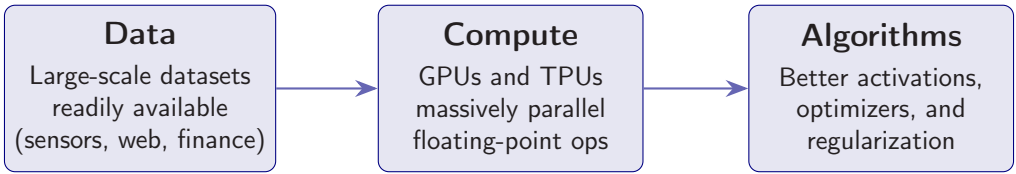
Structural models

- Solving and estimating DSGE, OLG and heterogeneous-agent models

This summer school focuses on the last category: deep learning as a *computational* tool for economic modeling.

Why Deep Learning Now?

Neural networks date back to the 1940s. Three developments made them practical:

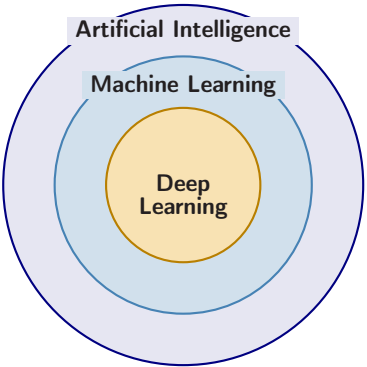


Key milestones: Perceptron (1958), Backpropagation (1986), Convolutional nets (LeCun et al. 1989; LeNet-5, 1998), AlexNet (2012), Transformers (2017), ChatGPT (2022).

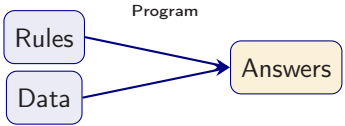
Part I

Machine Learning Foundations

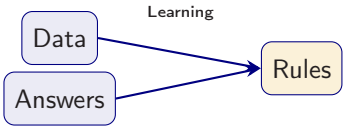
What Is Machine Learning?



Classical programming:



Machine learning:



Types of Machine Learning

Supervised	Unsupervised	Reinforcement
Given labeled pairs $\{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$, learn a mapping $h : \mathcal{X} \to \mathcal{Y}$.	Given only $\{\boldsymbol{x}^{(i)}\}_{i=1}^{m}$, discover hidden structure.	An agent acts in an environment, receives rewards, and learns a policy π.
Regression: $y \in \mathbb{R}$	**Clustering:** group similar points	**Policy:** $a_t = \pi(s_t)$
Classification: $y \in \{1, \ldots, K\}$	**Dim. reduction:** compress features	**Goal:** maximize $\sum_t \gamma^t r_t$

This morning we build up the *supervised* pipeline (model, loss, optimizer), then re-purpose it for the *self-supervised* method at the heart of Day 1, **Deep Equilibrium Nets**.

Supervised Learning: Regression

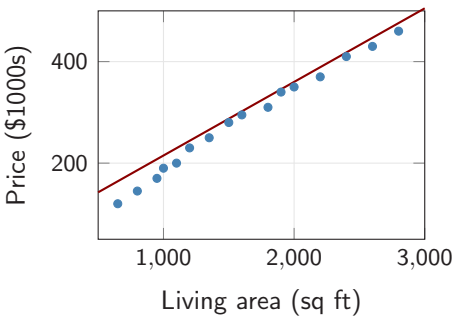
Goal: predict a continuous target $y \in \mathbb{R}$ from features $\boldsymbol{x} \in \mathbb{R}^d$.

Setup:

- Training data: $\{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$
- Model (hypothesis): $\hat{y} = h(\boldsymbol{x}; \boldsymbol{\theta})$
- Parameters $\boldsymbol{\theta}$ learned from data

Linear example:

$$h_{\boldsymbol{\theta}}(x) = \theta_0 + \theta_1 x$$



Supervised Learning: Classification

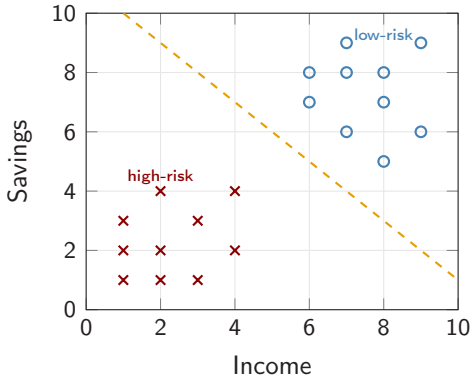
Goal: assign an input $\boldsymbol{x}$ to one of K classes.

Example, credit scoring:

- Features: income, savings
- Labels: low-risk / high-risk
- Decision boundary separates classes

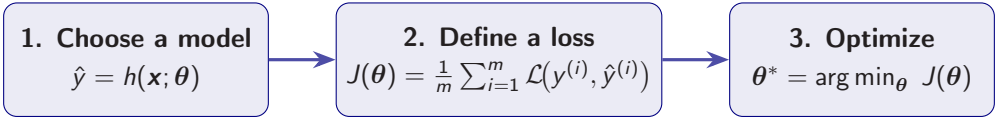
A simple linear rule:

$$\text{if } \boldsymbol{w}^\top \boldsymbol{x} + b > 0 \;\Rightarrow\; \text{low-risk}$$



The Machine Learning Recipe

Every machine learning algorithm follows the same three steps:



Cost function example (supervised regression: Mean Squared Error)

$$J(\boldsymbol{\theta}) \;=\; \frac{1}{2m} \sum_{i=1}^{m} \big(h_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)}) - y^{(i)}\big)^2$$

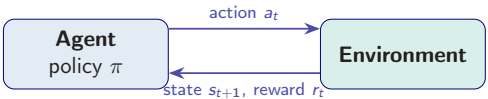
Same recipe, different loss. Deep Equilibrium Nets (next lecture) replace step 2 by an Euler-equation residual; steps 1 and 3 are unchanged.

Unsupervised and Reinforcement Learning (Brief)

Unsupervised learning: no labels, only inputs $\{\boldsymbol{x}^{(i)}\}_{i=1}^{m}$; discover hidden structure.

- **Clustering:** partition data into groups of similar points (customer segmentation).
- **Dimensionality reduction:** project to a lower-dimensional space (PCA, autoencoders).

Reinforcement learning: an agent learns a policy $\pi(a \mid s)$ to maximize $\sum_t \gamma^t r_t$.

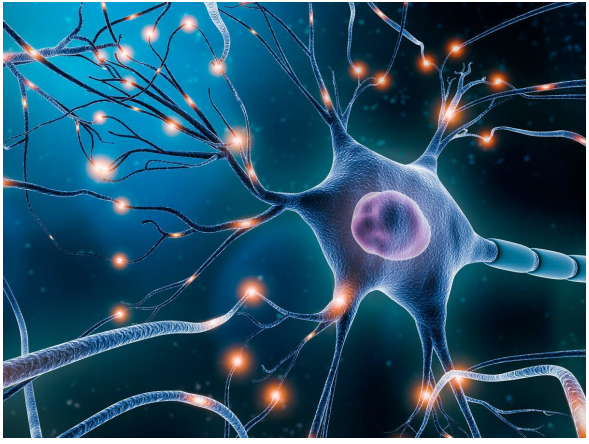


Deep RL for heterogeneous-agent models is covered this afternoon (Yang, 15:30). The neural network tools from this lecture are the building blocks for all of these paradigms.

Part II

From Perceptrons to Deep Networks

Biological Neurons



The brain is a massively parallel computer made of $\sim$86 billion neurons.

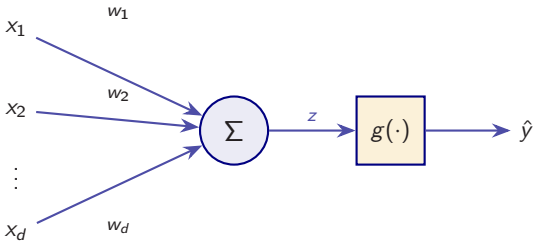
Each neuron:

- Receives signals via **dendrites**
- Integrates signals in the **cell body**
- Transmits output along the **axon**
- Communicates at **synapses**

Artificial neural networks are a *very loose* abstraction of this architecture.

The Artificial Neuron (McCulloch & Pitts, 1943)

From biology to math: synapses become weights, dendrites become inputs, integration becomes a sum, firing becomes a nonlinear activation.

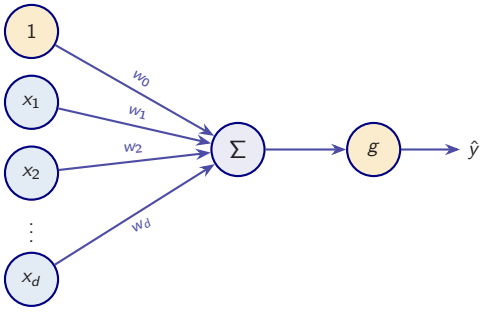


Three components:

1. **Weighted inputs** w_i (synaptic weights)
2. **Summation:** $z = \sum_{j=1}^{d} w_j x_j - \theta$ (threshold θ)
3. **Activation** $g(z)$: nonlinear function determining whether the neuron "fires"

The 1943 model fired via a step function, and its weights were *fixed*. Making them learnable came later (Hebb 1949; Rosenblatt 1958).

The Perceptron (Rosenblatt, 1958)



Forward pass:

$$\hat{y} \;=\; g\left(w_0 + \sum_{j=1}^{d} x_j\, w_j\right) \;=\; g\big(\boldsymbol{w}^\top \boldsymbol{x} + b\big)$$

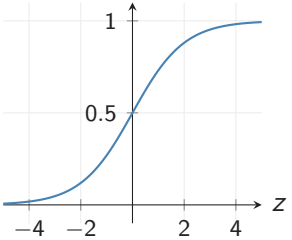
- $b \equiv w_0$ is the **bias** (shifts the activation)
- $g(\cdot)$ is the **activation function**. Rosenblatt used a **step**: output 1 if the sum is positive, else 0
- The weights are now learned from data, not fixed by hand

Without g this is just a linear model. The step has no useful derivative, so smooth activations come next.

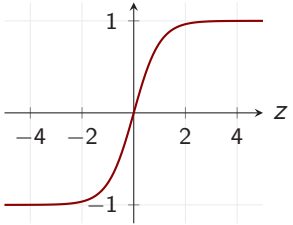
Minsky & Papert (1969): one perceptron cannot represent XOR, which is exactly why we need hidden layers.

Activation Functions

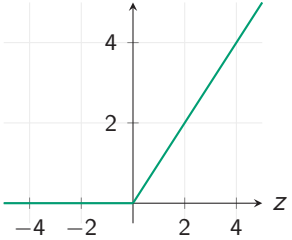
Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$



Tanh: $\tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$



ReLU: $\max(0, z)$



	Sigmoid	**Tanh**	**ReLU**	**Swish** $z\sigma(z)$ / **Softplus** $\ln(1+e^{z})$
Range	$(0, 1)$	$(-1, 1)$	$[0, \infty)$	$\approx$ ReLU, but smooth everywhere

Smoothness matters later: ReLU has $g'' = 0$, so if the loss contains *derivatives of the network* (Euler residuals, PDEs) use **tanh** or **Swish** instead.

The Output Layer Is a Modeling Choice

Hidden activations create flexibility. The output activation encodes what you *know* about the answer:

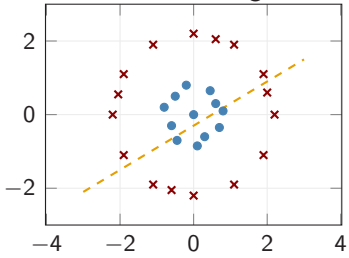
Output head	**Range**	**Use when**
Identity (none)	$\mathbb{R}$	unconstrained regression
Softmax	simplex	class probabilities
Softplus $\ln(1+e^z)$	$(0,\infty)$	must be positive (consumption)
Sigmoid $\sigma(z)$	$(0,1)$	a rate or share (savings rate)

Why this matters in the next lecture

Constraints built into the *architecture* hold exactly and cost nothing; a penalty in the loss holds only approximately. Deep Equilibrium Nets use a sigmoid savings head so $C_t > 0$ and $K_{t+1} > 0$ *before training starts*.

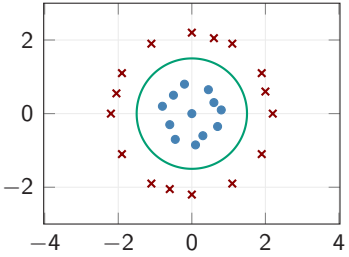
Why Nonlinearity Matters

Linear activation ⇒ straight boundary



no straight line can separate them

Nonlinear activation ⇒ curved boundary

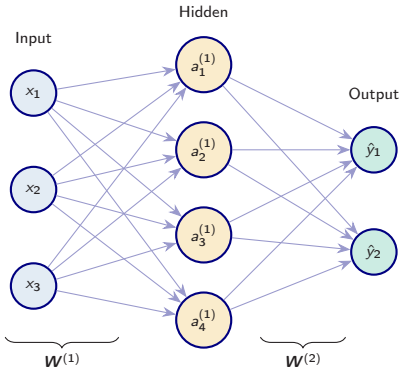


a curved boundary separates them exactly

Mathematical reason

A composition of linear maps is still linear: $\boldsymbol{W}_2(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2 = \boldsymbol{W}_2\boldsymbol{W}_1\boldsymbol{x} + (\boldsymbol{W}_2\boldsymbol{b}_1 + \boldsymbol{b}_2)$.
Without nonlinearities, depth is useless: no stack of linear layers can bend that circle.

Single Hidden Layer Network



Hidden layer:

$$z^{(1)} = W^{(1)}x + b^{(1)}$$
$$a^{(1)} = g(z^{(1)})$$

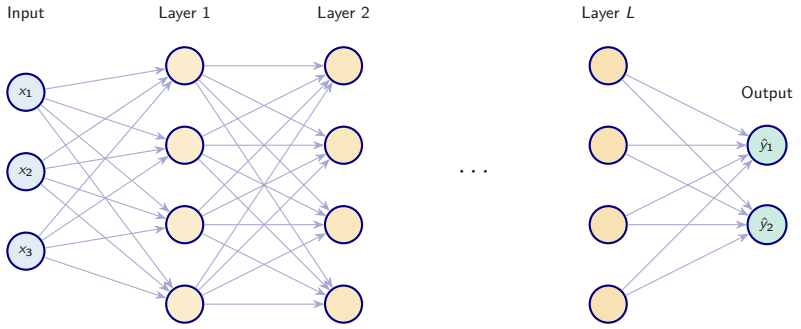
Output layer:

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$$
$$\hat{y} = g_{\text{out}}(z^{(2)})$$

where $W^{(1)} \in \mathbb{R}^{n_1 \times d}$, $W^{(2)} \in \mathbb{R}^{K \times n_1}$.

The hidden layer creates a **learned representation** of the input that the output layer then uses for prediction.

Deep Feedforward Networks



An L-layer network computes a nested composition:

$$\boldsymbol{a}^{(0)} = \boldsymbol{x}, \qquad \boldsymbol{a}^{(\ell)} = g_\ell\big(\boldsymbol{W}^{(\ell)}\boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}\big), \qquad f(\boldsymbol{x};\boldsymbol{\theta}) = \boldsymbol{a}^{(L)}$$

Each layer transforms the representation. For *specific* function classes, **depth** buys exponentially fewer parameters than width (Telgarsky 2016; Eldan & Shamir 2016).

Universal Approximation Theorem

Theorem (Cybenko 1989; Hornik, Stinchcombe & White 1989)

Let $g : \mathbb{R} \to \mathbb{R}$ be a bounded, non-constant, continuous activation function. Then for any continuous function $f : \mathcal{K} \to \mathbb{R}$ on a compact set $\mathcal{K} \subset \mathbb{R}^d$ and any $\varepsilon > 0$, there exists a single-hidden-layer network

$$F(\boldsymbol{x}) = \sum_{j=1}^{n_1} v_j \, g(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j)$$

such that $\sup_{\boldsymbol{x} \in \mathcal{K}} |F(\boldsymbol{x}) - f(\boldsymbol{x})| < \varepsilon$.

Caveats:

- The required width n_1 may be **exponentially large** in d, and existence $\neq$ efficient learnability.
- Practical success suggests **depth** is a far more efficient parameterization than width alone.
- Boundedness can be dropped: density holds iff the activation is **non-polynomial** (Leshno et al., 1993), and this is what covers **ReLU**.

Why Networks and Not Grids? The Curse of Dimensionality

A grid dies exponentially. Tabulating a function with N points per dimension costs

$$N^d \quad \text{points.}$$

With $N = 11$: $d = 2 \to 121$;
$d = 5 \to 1.6 \times 10^5$; $d = 10 \to 2.6 \times 10^{10}$.

A network does not. Training a net with P parameters on M sampled points costs $O(MP)$, and neither has to grow like N^d.

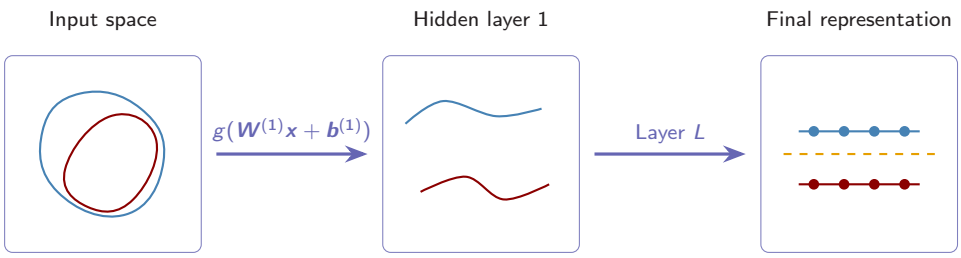
Barron (1993)

For functions in the *Barron class*, a one-hidden-layer network with n_1 units achieves squared error $O(1/n_1)$, with a constant that does not depend on d.

Be honest: the Barron class is restrictive and d can hide in the constant. A strong *hint*, not a free lunch, but it is why we sample instead of tabulate.

A Geometric View: Learning to Untangle

Neural networks learn **successive coordinate transformations** that make the data linearly separable.



Analogy (Chollet, 2017): imagine crumpling two sheets of colored paper into a ball. A neural network learns how to smooth them out again, layer by layer, until a simple linear classifier can separate the colors.

Part III

Training: Loss, Gradients, and Optimization

Loss Functions

The loss function $J(\boldsymbol{\theta})$ measures how far predictions $\hat{\boldsymbol{y}}$ are from targets $\boldsymbol{y}$.

Regression: Mean Squared Error

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^{n} \left(y^{(i)} - f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}) \right)^2$$

Penalizes large deviations quadratically.

Classification: Cross-Entropy

$$J = -\frac{1}{n} \sum_{i=1}^{n} \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

For binary labels $y \in \{0, 1\}$.

Multiclass (K classes): use softmax output $\hat{y}_k = e^{z_k} / \sum_j e^{z_j}$ and categorical cross-entropy: $J = -\frac{1}{n} \sum_i \sum_k y_k^{(i)} \log \hat{y}_k^{(i)}$.

Choosing a Loss Function

Beyond MSE, with residual $r = y - \hat{y}$: **Huber** is quadratic near zero and linear in the tails, and the **quantile** (pinball) loss $\rho_\tau(r) = r\,(\tau - 1[r < 0])$ is deliberately asymmetric.

Loss	Sensitivity to outliers	Symmetry	Use case
MSE	High	Symmetric	Default regression
Huber	Low	Symmetric	Robust regression
Quantile	Low	**Asymmetric**	Risk, VaR, tails

Why this matters in economics and finance

Tail risks are often more important than average performance. **Quantile loss** lets the model focus on getting worst-case predictions right, directly producing **Value-at-Risk** estimates. (Expected Shortfall is *not* elicitable on its own; it needs the joint VaR/ES scoring function of Fissler & Ziegel, 2016.)

References: Huber (1964), Koenker & Bassett (1978).

Gradient Descent: The Idea

Goal: $\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$. Step downhill from a random start, with learning rate $\eta > 0$:

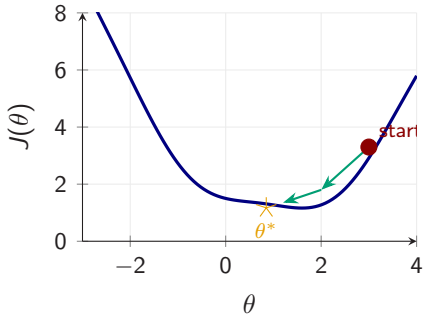
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}).$$

Algorithm

Initialize $\boldsymbol{\theta} \sim \mathcal{N}(0, \sigma^2 \boldsymbol{I})$, then repeat:

$$\boldsymbol{g} \leftarrow \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \boldsymbol{g}.$$

Each step costs $O(N)$ over the whole training set, and the iteration converges to a **stationary point**, usually a *saddle* (Dauphin et al. 2014).



Stochastic Gradient Descent (SGD)

Idea: approximate the full gradient with a cheap estimate from a small subset of data.

Single-sample SGD

Pick one random example $(\boldsymbol{x}^{(i)}, y^{(i)})$:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \nabla_{\boldsymbol{\theta}} J_i(\boldsymbol{\theta})$$

Cheap but noisy.

Mini-batch SGD

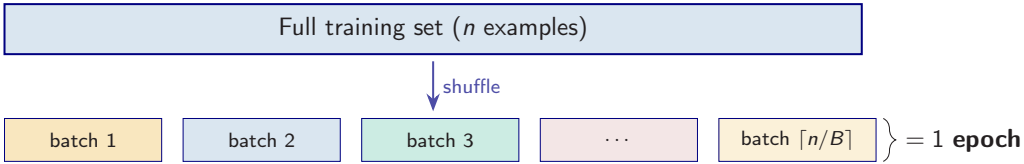
Sample a batch $\mathcal{B}$ of B examples:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{B} \sum_{k \in \mathcal{B}} \nabla_{\boldsymbol{\theta}} J_k(\boldsymbol{\theta})$$

Reduces variance; enables GPU parallelism.

Why does this work? The mini-batch gradient is an **unbiased estimate** of the full gradient, $\mathbb{E}\big[\frac{1}{B}\sum_{k\in\mathcal{B}} \nabla J_k\big] = \nabla J$.
Unbiasedness *plus* a decaying step size, $\sum_t \eta_t = \infty$ and $\sum_t \eta_t^2 < \infty$, gives convergence (Robbins & Monro, 1951). With a constant η, SGD instead hovers in an $O(\eta)$ neighbourhood of the optimum, hence learning-rate decay.

Mini-Batches and Epochs

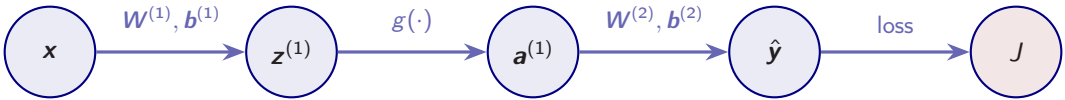


- One **epoch** = one complete pass through the training set.
  - When data are *simulated* rather than fixed, as in the next lecture, there is no epoch: you draw fresh mini-batches forever.
- Typical batch sizes: $B \in \{32, 64, 128, 256\}$.
- Benefits of mini-batches:
  - Smoother convergence than single-sample SGD.
  - Efficient use of GPU parallelism.
  - Implicit regularization from gradient noise.

Backpropagation: The Key Idea

Problem: how do we compute $\frac{\partial J}{\partial \boldsymbol{W}^{(\ell)}}$ for each layer ℓ?

Answer: the **chain rule** of calculus, applied systematically.



$$\frac{\partial J}{\partial \boldsymbol{W}^{(1)}} = \underbrace{\frac{\partial J}{\partial \hat{\boldsymbol{y}}}}_{\text{output error}} \cdot \underbrace{\frac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{a}^{(1)}}}_{\boldsymbol{W}^{(2)}} \cdot \underbrace{\frac{\partial \boldsymbol{a}^{(1)}}{\partial \boldsymbol{z}^{(1)}}}_{g'(\boldsymbol{z}^{(1)})} \cdot \underbrace{\frac{\partial \boldsymbol{z}^{(1)}}{\partial \boldsymbol{W}^{(1)}}}_{\boldsymbol{x}}$$

Backpropagation: The Backward Pass

Forward pass first: evaluate $\boldsymbol{z}^{(\ell)}, \boldsymbol{a}^{(\ell)}$ layer by layer and **store them**, because the backward pass needs them.

Define the **error signal** at layer ℓ: $\boldsymbol{\delta}^{(\ell)} \equiv \partial J / \partial \boldsymbol{z}^{(\ell)} \in \mathbb{R}^{n_\ell}$.

Output layer ($\ell = L$):

$$\boldsymbol{\delta}^{(L)} = \nabla_{\boldsymbol{z}^{(L)}} J = \frac{\partial J}{\partial \hat{\boldsymbol{y}}} \odot g'_{\text{out}}(\boldsymbol{z}^{(L)})$$

Hidden layers ($\ell = L-1, \ldots, 1$), propagating the error *backward*:

$$\boxed{\boldsymbol{\delta}^{(\ell)} = \left(\boldsymbol{W}^{(\ell+1)}\right)^\top \boldsymbol{\delta}^{(\ell+1)} \odot g'\left(\boldsymbol{z}^{(\ell)}\right)}$$

Parameter gradients:

$$\frac{\partial J}{\partial \boldsymbol{W}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} \left(\boldsymbol{a}^{(\ell-1)}\right)^\top, \qquad \frac{\partial J}{\partial \boldsymbol{b}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)}$$

$\odot$ is the element-wise product. The form of $\boldsymbol{\delta}^{(L)}$ above assumes a *coordinatewise* g_{out}; for **softmax** + **cross-entropy** (and linear + MSE) it collapses to $\boldsymbol{\delta}^{(L)} = \hat{\boldsymbol{y}} - \boldsymbol{y}$.

Backpropagation Is Automatic Differentiation

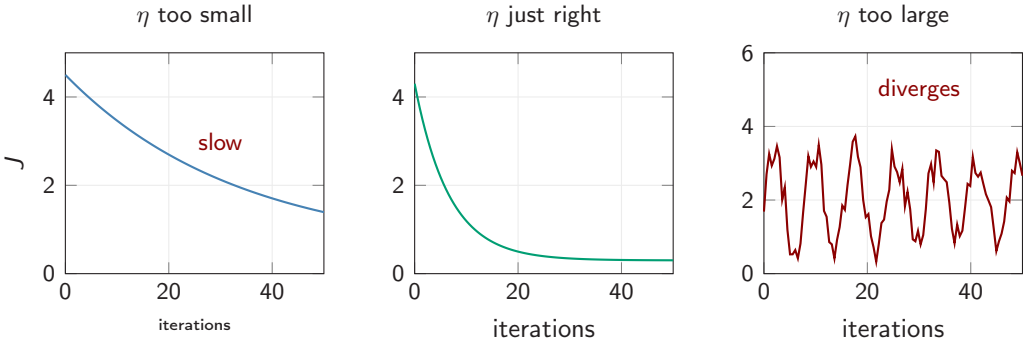
You will never implement the previous slide by hand. Backpropagation is exactly reverse-mode automatic differentiation, and every framework does it for you.

- TensorFlow / PyTorch / JAX record the **computation graph** as you evaluate the forward pass, then walk it backwards.
- They return $\nabla_{\boldsymbol{\theta}}$ of any scalar you can build from $\boldsymbol{\theta}$, and the code does not care where that scalar came from.
- Cost: one backward pass $\approx$ 2–3$\times$ one forward pass, **independent of the number of parameters**.

The hinge of this whole summer school

Nothing in autodiff requires the loss to come from *labels*; it only has to be a differentiable function of $\boldsymbol{\theta}$. So we can put an Euler-equation residual in the loss and get $\nabla_{\boldsymbol{\theta}}$ for free. That is exactly what **Deep Equilibrium Nets** do next.

The Learning Rate



- **Too small:** slow convergence, may get stuck in poor local minima.
- **Too large:** oscillation or divergence, overshooting the minimum.
- **Solution:** use **adaptive learning rates** that adjust automatically.

Adaptive Optimizers

Plain SGD uses one learning rate for every parameter. **Momentum** averages the gradient over time to damp oscillations. **Adam** goes further and gives each coordinate its own step.

Adam (Kingma & Ba, 2015)

Track a first and a second moment, correct their bias, then rescale:

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\nabla J, \qquad \boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1-\beta_2)(\nabla J)^2,$$

$$\hat{\boldsymbol{m}}_t = \boldsymbol{m}_t/(1-\beta_1^t), \quad \hat{\boldsymbol{v}}_t = \boldsymbol{v}_t/(1-\beta_2^t), \qquad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\boldsymbol{m}}_t}{\sqrt{\hat{\boldsymbol{v}}_t} + \varepsilon}$$

What to actually do

Use **Adam** with the defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$; tune only η, starting at 10^{-3}.

RMSProp is the same idea using only the second moment; for true weight decay, use **AdamW**.

Weight Initialization

Why not initialize all weights to zero? Every neuron would compute the same output, receive the same gradient, and the symmetry would never break.

Xavier / Glorot (2010): sigmoid, tanh

$$W_{ij}^{(\ell)} \sim \mathcal{N}\left(0,\ \frac{2}{n_{\ell-1}+n_{\ell}}\right)$$

He (2015): ReLU

$$W_{ij}^{(\ell)} \sim \mathcal{N}\left(0,\ \frac{2}{n_{\ell-1}}\right)$$

Both keep activation variance roughly constant across layers; biases start at zero.

Standardize your inputs

These variances are *derived* assuming zero-mean, unit-variance inputs. An economic state vector mixing capital, TFP and portfolio shares silently breaks that.

Vanishing and Exploding Gradients

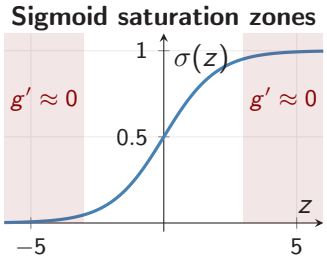
The backward pass multiplies a factor through every layer,

$$\delta^{(\ell)} = \left(\boldsymbol{W}^{(\ell+1)}\right)^{\top} \delta^{(\ell+1)} \odot g'(\boldsymbol{z}^{(\ell)}),$$

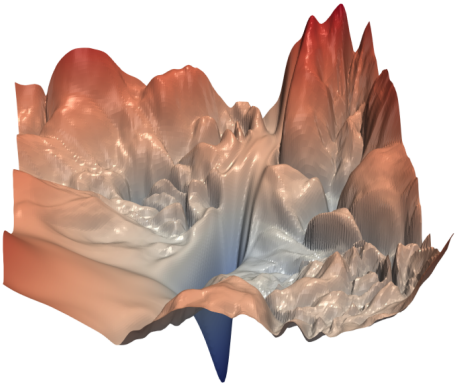
so the per-layer growth factor is $\|\boldsymbol{W}^{(\ell+1)}\| \cdot |g'(\boldsymbol{z}^{(\ell)})|$.

- Sigmoid has $g' \leq \frac{1}{4}$, so with $\|\boldsymbol{W}\| \approx 1$ the product decays like 4^{-L}, the **vanishing gradient** case.
- Large $\|\boldsymbol{W}\|$ gives the opposite: **exploding gradients**.

Remedies: ReLU, scaled initialization, **residual connections** (He et al. 2016), batch norm, gradient clipping.



Loss Landscapes of Neural Networks



The loss surface of a deep network is highly non-convex:

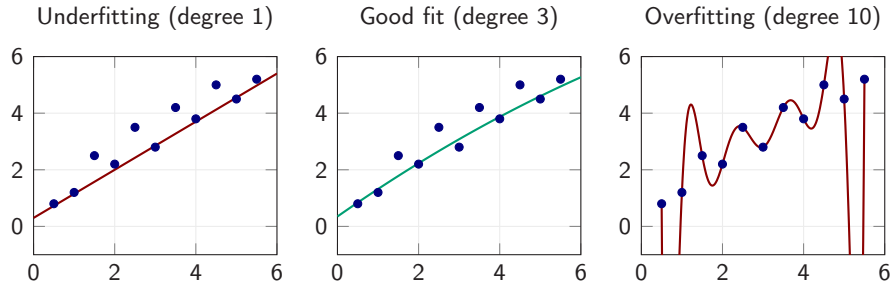
- Many **local minima** and **saddle points**
- SGD noise helps escape poor ones

Li et al. (2018) *visualize* these surfaces. A widely-held *hypothesis* says flat minima generalize better (Hochreiter & Schmidhuber 1997; Keskar et al. 2017), but sharpness is reparameterization-dependent (Dinh et al. 2017), so treat it as intuition rather than theorem.

Part IV

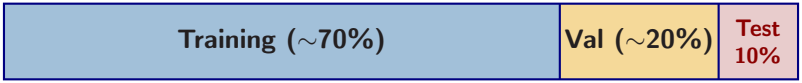
Generalization and Regularization

Overfitting and Underfitting



- **Underfitting:** model too simple, high bias, cannot capture the pattern.
- **Overfitting:** model too complex. It passes through *every* training point but swings wildly between them, so predictions off the sample are useless.
- **Goal:** find the right complexity that generalizes well.

Train / Validation / Test Split



Training set

Used to fit model parameters θ via optimization.

Validation set

Used for model selection: hyperparameters, architecture, early stopping.

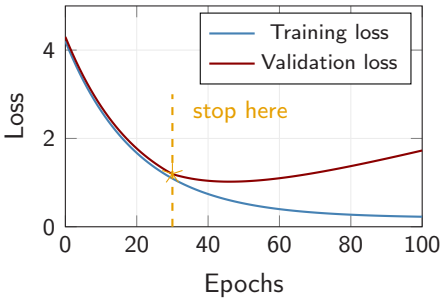
Test set

Touched **once** at the very end to report final performance.

Golden rule

Never use the test set for any decision during model development.

Early Stopping



Idea: monitor the validation loss; stop when it starts rising.

In practice:

1. Evaluate the validation loss after each epoch.
2. Save the weights whenever it improves.
3. Stop after p epochs without improvement ("patience"); return the saved best weights.

It is the simplest effective regularizer, and it is essentially free. Other standard options: an L_2 **penalty**, **dropout**, **data augmentation**.

Regularization: The L_2 Penalty

Add a penalty on the size of the weights (usually excluding biases):

$$\tilde{J}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2$$

Large weights are what let a network produce wild, wiggly functions. Shrinking them biases the learner toward smoother hypotheses.

λ is chosen on the **validation** set.

$L_2 \neq$ weight decay under Adam

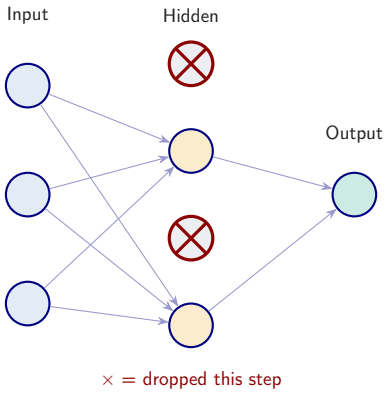
For plain SGD the two are equivalent. For **Adam** they are not, because the penalty gradient gets divided by $\sqrt{\hat{\boldsymbol{v}}_t}$ along with everything else.

Use **AdamW** if you want true decay (Loshchilov & Hutter, 2019).

Regularization: Dropout

Dropout (Srivastava et al., 2014): during *training*, independently zero each hidden unit with probability p (typically 0.5).

- The network cannot rely on any single unit, so it must build redundant representations.
- At *test* time all units are active, with no dropping.
- Equivalent to training an **ensemble** of sub-networks that share weights.



Part V

Outlook, Software, and Exercises

Beyond Feedforward Networks: Sequences and Attention

Feedforward nets are *memoryless*: $\hat{y}_t = f(\boldsymbol{x}_t)$. For sequential data, three families add memory.

RNN

A hidden state $\boldsymbol{h}_t$ carries a running summary of the past.

LSTM / GRU

Gates protect a memory lane, so signals survive many steps.

Transformer

Attention lets every position read every other, in parallel.

Rule of thumb

Specialized pattern, moderate history $\Rightarrow$ an LSTM is still a strong baseline. Long context and parallelism matter $\Rightarrow$ start with a Transformer.

Software Ecosystem

TensorFlow
+ Keras
Google, 2015

PyTorch
Meta AI, 2017

JAX
Google, 2018

- **Keras** is a high-level API that runs on top of TensorFlow, PyTorch, or JAX, so you write once and run anywhere.
- Documentation and tutorials for all three are linked from the course repository.

This morning

We use **TensorFlow** / **Keras**. A first network is about ten lines of code.

What Comes Next, and What to Run

11:00 – 12:30: Deep Equilibrium Nets (part II)

Everything from this lecture (networks, losses, SGD, regularization) comes together when we train a neural network to solve a dynamic stochastic model.

Warm-up exercise (notebook 01_06): approximate the Genz (1987) test functions with a neural network.

- Train networks on 10, 50, 100, 500 random points from $[0,1]^d$ for $d \in \{2, 5, 10\}$.
- Vary the **depth** (1–5 hidden layers), the **activation** (tanh, ReLU, Swish), and the **optimizer** (SGD, Adam).
- Plot test error vs. the number of training points, and watch the curse of dimensionality.

Notebooks 01_01 to 01_05 cover the rest: basic ML, gradient descent and SGD, double descent, a first deep network, and the same tasks in PyTorch.

Materials: course GitHub repository · **Platform:** nuvolos.cloud (no local setup required)

Questions?

Simon Scheidegger

HEC, University of Lausanne

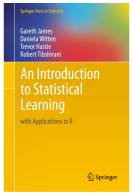
https://sischei.github.io

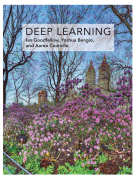
Materials:

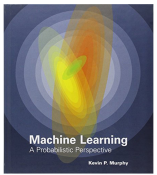
github.com/sischei/summer-school-AI-for-economics-and-finance-2026

Backup slides

Useful Materials







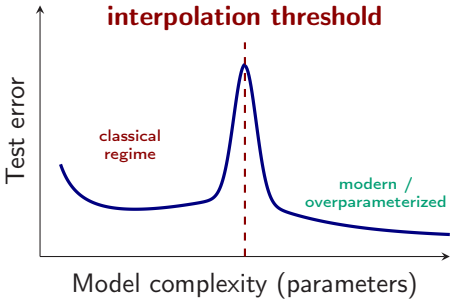
1. **S. J. D. Prince** (2023).
   Understanding Deep Learning. MIT Press.
   (free PDF)

2. **I. Goodfellow, Y. Bengio, A. Courville** (2016).
   Deep Learning. MIT Press. (free online)

3. **K. P. Murphy** (2022).
   Probabilistic Machine Learning: An Introduction. MIT Press. (free PDF)

4. **G. James et al.** (2021).
   An Introduction to Statistical Learning.
   Springer. (free PDF)

Deep Double Descent



The classical U turns into a **second descent** once the model is overparameterized. Three regimes along the curve:

1. Classical ($n_{\text{par}} < n_{\text{data}}$): the familiar bias–variance U: small models underfit, larger ones chase noise.

2. Interpolation peak ($n_{\text{par}} \approx n_{\text{data}}$): model *just* fits the training set; among all zero-error solutions the optimizer's pick is unstable $\Rightarrow$ variance explodes.

3. Modern ($n_{\text{par}} \gg n_{\text{data}}$): GD appears to select a low-complexity, minimum-norm-like interpolator, provable for linear/kernel models but conjectural for deep nets.

Caveat: the peak is often absent under proper regularization, and more *data* can also hurt near the threshold (Nakkiran et al. 2019).

Belkin et al. (2019); Nakkiran et al. (2019).